

第 1 次课实验指导书：从 0 启动 LoongArch miniOS

技术编号：week01 | 建议检查点：week01-qemu-hello
称“第 1 次课”，不称“第 1 周”。配合讲义：docs/week01/lecture_notes.md。

1. 实验目标

完成本次课最小可验证闭环，主题：从 0 启动 LoongArch miniOS。

预期/产出：

```
Hello miniOS on LoongArch64
```

2. 实验环境与准备

- 推荐：WSL Ubuntu / Linux + LoongArch 交叉工具链 + QEMU
- 基本命令：make、loongarch64-linux-gnu-gcc、qemu-system-loongarch64
- 仓库根目录执行实验命令
- 重点文件：
 - boot/start.S
 - kernel/main.c
 - kernel/printk.c
 - include/uart.h

通用准备：

```
cd <miniOS 仓库根目录>
git status
# 按本次课 tag 切换（若已发布）
```

3. 实验任务

Task1 环境检查

- 确认可执行：make、交叉编译 gcc、qemu-system-loongarch64。
- 记录本机检查命令与输出摘要。

Task2 获取代码

- git fetch --tags
- git switch -c my-week01-lab week01-qemu-hello

Task3 编译运行

- make clean && make && make run
- 记录完整串口输出；退出 QEMU：Ctrl-a 后按 x。

Task4 阅读启动路径

- 在 boot/start.S 标出 _start、设栈、调用 kernel_main、halt。
- 回答三问：第一条指令在哪？为何不是 main？为何不能 printf？

Task5 (选做) 检查点对比

- 依次观察 week01-00-skeleton ...week01-03-printk-uart 的输出差异。

4. 验收标准

- 真实运行得到一行: Hello miniOS on LoongArch64
- 实验报告含命令与真实输出, 不得编造
- 能口述 CPU → _start → kernel_main → UART 路径

5. 实验报告要求

报告至少包含:

1. 环境说明 (OS、工具版本摘要)
2. 关键命令与**真实输出** (可截断, 但不可编造)
3. 对核心代码/指令的简要注释或流程图
4. 问题与解决过程 (若有)
5. 思考题作答

6. AI 共学边界

仅允许解释与画图, 不允许直接生成实验代码。

7. 思考题

1. 不设 \$sp 可能发生什么?
2. printk 与用户态 printf 差在哪一层?
3. 为什么先 QEMU 再开发板?

8. 提交清单

- ☐ 实验报告 (PDF/Markdown)
- ☐ 关键输出摘录
- ☐ 需要提交的代码补丁或笔记 (按教师要求)